

Toxic-Query Classification with a Simple RNN

A multi-class text-classification pipeline for content-safety triage

Notebook	<code>rnn.ipynb</code>
Task	Multi-class toxic-content classification (9 categories) using a SimpleRNN
Dataset	<code>cellula_toxic_data.csv</code> (3,000 raw rows)
Author	Eslam Mohamed Fawzy
Date	9 July 2026
Framework	TensorFlow / Keras, scikit-learn, NLTK

Abstract

This notebook builds an end-to-end Natural Language Processing pipeline that classifies short user queries (paired with an image description) into one of nine content-safety categories such as *Safe*, *Violent Crimes*, and *Non-Violent Crimes*. The raw text is cleaned, tokenised into integer sequences, padded to a fixed length, and fed to a word-*Embedding* layer followed by a *SimpleRNN* and a softmax classifier. The model is evaluated primarily with the **F1-score** because the class distribution is severely imbalanced. The trained model reaches a **weighted F1 of 0.753**, satisfying the assignment threshold of $F1 \geq 0.75$.

1 Objective

The goal of this notebook is to train a supervised **text classifier** that reads a short natural-language request and predicts which **content-safety category** it belongs to. This is a *multi-class* problem with nine mutually-exclusive labels. Concretely, given a sentence such as “How do I go about selling stolen art pieces?” the model must output the label *Violent Crimes* (or the correct category), while an innocuous sentence such as “What are the health benefits of daily exercise?” must be routed to *Safe*.

Why the task matters

Automated content-safety triage is a core building block of modern AI systems. Before a large language model answers a user, a lightweight guardrail classifier can flag potentially harmful requests (violence, self-harm, sexual exploitation, election interference, ...) so they can be blocked, escalated, or handled with special care. A fast, well-calibrated classifier reduces the risk of an AI assistant producing harmful content and lowers the human-moderation workload.

NLP concepts exercised

Concept	Where it is used in the notebook
Text normalisation / cleaning	lower-casing, contraction expansion, punctuation & URL removal
Tokenisation & vocabulary building	Keras <code>Tokenizer</code> with an out-of-vocabulary token
Sequence encoding	words → integer indices → fixed-length padded vectors
Distributed word representations	trainable <code>Embedding</code> layer (128-d word vectors)
Sequence modelling	<code>SimpleRNN</code> reads the tokens left-to-right and keeps a hidden state
Imbalanced-class evaluation	weighted & macro F1-score , per-class precision/recall

DEEP-LEARNING MODEL

The trained network is a **Recurrent Neural Network** of the vanilla / “simple” type (Keras `SimpleRNN`). It processes the sentence one token at a time, maintaining a single 128-dimensional hidden state that is updated at every step and finally used to predict the category.

2 Dataset Description

The data lives in `Data/cellula_toxic_data.csv` and is loaded with pandas. Each row is a labelled example with three columns.

Column	Meaning	Type	Distinct values (raw)
<code>query</code>	the user’s natural-language request (the main signal)	text	2,009 unique of 3,000

image descriptions	a short caption of an accompanying image	text	only 12 unique templates
Toxic Category	target label — one of 9 safety classes	categorical	9 classes

Size and integrity

- **Raw size:** 3,000 rows × 3 columns.
- **Missing values:** none — `df.isnull().sum()` returns 0 for every column, so no imputation is required.
- **Duplicates:** 973 fully-duplicated rows were found and removed, leaving **2,027 unique rows**. Duplicates are common here because both the queries and the 12 image captions repeat; leaving them in would let identical rows leak between train and test and inflate the score.

Class distribution and the imbalance problem

After de-duplication, the nine labels are distributed as shown below. This is the single most important property of the dataset and it dictates every later modelling decision.

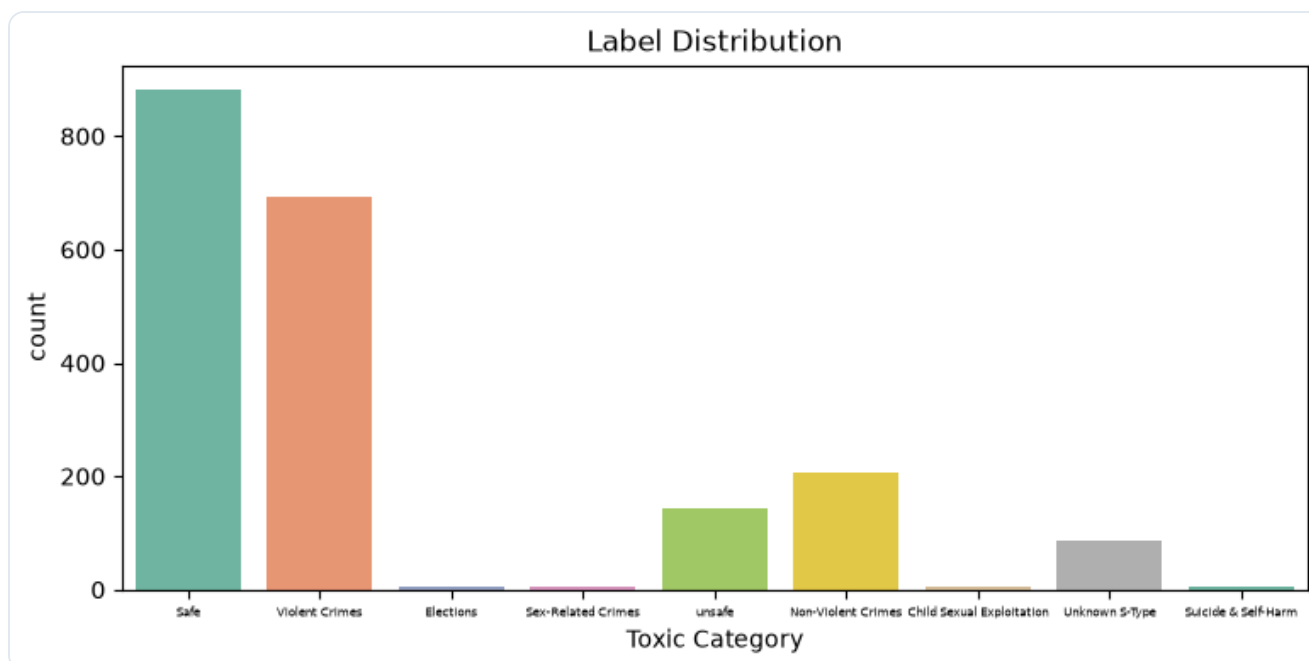
Class	Count	Share
Safe	881	43.5 %
Violent Crimes	693	34.2 %
Non-Violent Crimes	207	10.2 %
unsafe	143	7.1 %
Unknown S-Type	86	4.2 %
Suicide & Self-Harm	5	0.25 %
Elections	4	0.20 %
Sex-Related Crimes	4	0.20 %
Child Sexual Exploitation	4	0.20 %

SEVERE CLASS IMBALANCE

The two largest classes (*Safe* + *Violent Crimes*) account for **~78 %** of the data, while four classes have **fewer than six examples each**. A naive model could reach ~43 % accuracy simply by always predicting *Safe*. This is precisely why **accuracy alone is misleading** and why the assignment mandates the F1-score (see §10).

3 Exploratory Data Analysis (EDA)

3.1 Label distribution plot



Count of examples per Toxic Category (after de-duplication). Produced by the notebook with `sns.countplot`.

What it shows: the number of rows per class. **Why it is useful:** it makes the imbalance visually obvious — two tall bars dominate and five bars are almost invisible. **Conclusion:** the model will see plenty of *Safe/Violent Crimes* examples but almost none of the rare classes, so we should expect strong performance on the majority classes and poor recall on the minority ones. This prediction is confirmed later by the per-class F1 scores.

3.2 Text-length statistics

The notebook measures the number of whitespace-separated words in each field with `str.split().str.len()`. Understanding sequence length is essential for choosing the padding length `maxlen`.

Field	Mean	Std	Min	Median	Max
<code>query</code> length	13.7	8.5	2	12	140
<code>image descriptions</code> length	8.2	1.4	6	8	11
<code>combined</code> (query + [SEP] + image)	22.9	8.7	10	21	152

Interpretation: queries are short (a median of 12 words) but have a very long right tail (max 140 words). Image captions are almost constant in length because they are drawn from only 12 templates. The heavy tail means a small number of very long sentences exist; padding every sentence to the maximum length would waste memory and add noise, so the notebook instead uses a percentile-based length (see §5).

3.3 Vocabulary size

After cleaning and tokenising, the corpus contains **4,500 distinct words** (`vocab_size = 4,501` including the padding index 0). This is a small, manageable vocabulary, which is appropriate for a from-scratch trainable embedding on a few-thousand-row dataset.

4 Data Preprocessing

This is the heart of the NLP pipeline. Each step converts raw human text into the fixed-shape numeric tensor a neural network requires. The steps are explained in the order they appear in the notebook.

4.1 De-duplication

`df.drop_duplicates()` removed 973 identical rows. **Why:** duplicated rows bias the class frequencies and, more dangerously, can place the *same* example in both the training and test sets, producing an over-optimistic score (data leakage).

4.2 The “V-stack” data-augmentation trick

Instead of feeding only the `query`, the notebook **vertically stacks** the `query` column and the `image descriptions` column into a single `text` column, each keeping the same label:

```
# query rows and image-description rows are concatenated
stacked_df = pd.concat([query_df, image_df], ignore_index=True)
```

This doubles the number of training rows from 2,027 to **4,054**. **Why it helps:** it gives the embedding layer more text to learn word vectors from and treats the image caption as an additional weak signal for the same label. **Trade-off:** the 12 repeated captions are only loosely related to toxicity, so they inject label noise — a caption like “*A child playing in a sunny meadow*” is tagged with whatever category its original row had, which is not always *Safe*. This is a genuine weakness discussed in §13.

4.3 Text cleaning

The `clean_text()` function normalises every string. Each operation removes a source of spurious vocabulary so that words that mean the same thing map to the same token.

Step	Purpose (NLP reasoning)
<code>text.lower()</code>	“Kill” and “kill” should be one token, not two; halves effective vocabulary noise.
Remove HTML <code><.*?></code>	strips markup that carries no semantic meaning.
Remove URLs	links are near-unique strings that would explode the vocabulary with useless tokens.
<code>contractions.fix</code>	expands “what’s” → “what is” so contractions match their expanded forms.
Remove punctuation	drops <code>?,.!</code> etc. which add no class signal for this task.
Collapse whitespace	normalises spacing so tokenisation is consistent.

VERIFIED EFFECT

The notebook prints five before/after examples. E.g. “*What’s the best way to keep criminal elements out of your workplace?*” becomes “*what is the best way to keep criminal elements out of your workplace*” — lower-cased, contraction expanded, punctuation gone. This is exactly the intended behaviour.

Note on stop-words / stemming: the notebook deliberately does *not* remove stop-words or apply stemming/lemmatisation. For a neural sequence model this is a reasonable choice — the embedding + RNN can learn which words matter, and words like “*not*” or “*how*” (often treated as stop-words) actually carry intent signal for safety classification.

4.4 Tokenisation and vocabulary creation

```
tokenizer = Tokenizer(oov_token="<OOV>")
tokenizer.fit_on_texts(texts)
vocab_size = len(tokenizer.word_index) + 1 # = 4501
```

The `Tokenizer` scans all text, builds a word→integer dictionary ordered by frequency (so `the`, `a`, `to` get the smallest indices), and reserves index 1 for the `<OOV>` token. **Why OOV matters:** any word seen at test time that was not in the training vocabulary maps to `<OOV>` instead of crashing or being silently dropped. Index 0 is kept free for padding, hence `vocab_size = 4,500 + 1`.

4.5 Converting words to integer sequences

`texts_to_sequences` replaces each word by its index, e.g. “*what is the right way ...*” → `[16, 11, 6, 224, 84, ...]`. The sentence is now a variable-length list of integers.

4.6 Choosing the padding length

Neural networks need fixed-size input, so every sequence must be the same length. The notebook analyses the sequence-length distribution to pick a sensible cut-off rather than guessing:

Percentile	25th	50th	75th	90th	95th	Max
Length (tokens)	7	10	12	17	21	147

WHY MAXLEN = 21

The 95th percentile is 21 tokens. Choosing `maxlen = 21` keeps **95 % of sentences fully intact** while truncating only the extreme 5 % tail (up to 147 tokens). Padding to the max of 147 would make 95 % of the tensor be zeros — wasted computation and diluted signal. This is a textbook length-vs-coverage trade-off.

4.7 Padding & truncation

```
X = pad_sequences(sequences, maxlen=21,
                  padding="post", truncating="post")
# X.shape = (4054, 21)
```

`padding="post"` appends zeros *after* the sentence; `truncating="post"` cuts overly-long sentences at the end. The result is a clean **4,054 × 21** integer matrix — one row per example, 21 columns per row.

4.8 Label encoding

```
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(stacked_df["label"])
```

`LabelEncoder` maps the nine string labels to integers 0–8 *alphabetically* (*Child Sexual Exploitation*=0, ..., *Violent Crimes*=7, *unsafe*=8). Integer labels are required by the `sparse_categorical_crossentropy` loss used later, which avoids having to one-hot encode the targets.

4.9 Train / test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

An 80/20 split yields **3,243 training** and **811 test** examples. **Crucially**, `stratify=y` keeps each class's proportion identical in both splits — without it, the four 4-example classes could vanish entirely from the test set. `random_state=42` makes the split reproducible.

Table — Preprocessing pipeline at a glance

#	Step	Input → Output
1	Drop duplicates	3,000 → 2,027 rows
2	V-stack query + image caption	2,027 → 4,054 rows
3	Clean text	raw string → normalised string
4	Tokenise	string → integer list (vocab 4,501)
5	Pad / truncate	variable list → length-21 vector
6	Label-encode	9 strings → ints 0–8
7	Stratified split	4,054 → 3,243 train / 811 test

5 Feature Engineering — Word Embeddings

Classical NLP represents text with sparse count vectors such as **Bag-of-Words** or **TF-IDF**. This notebook instead uses a **trainable embedding**, which is the modern deep-learning approach.

Representation	Idea	Pros	Cons
Bag-of-Words	count of each word	simple, interpretable	huge sparse vector, ignores word order & meaning
TF-IDF	counts re-weighted by rarity	down-weights common words	still sparse, still order-blind
Embedding (used here)	each word → dense 128-d learned vector	captures similarity, low-dimensional, order preserved when fed to an RNN	needs data to train, less interpretable

MATHEMATICAL INTUITION

The embedding is a lookup table E of shape (4501×128) . Token id t selects row $E_t \in \mathbb{R}^{128}$. During training, back-propagation nudges the vectors so that words used in similar toxic contexts ("kill", "attack", "bomb") end up near each other in the 128-dimensional space. This dense, semantic representation is what makes the downstream RNN effective on only a few thousand examples.

6 Model Architecture

The model is a Keras `Sequential` stack of three layers. The notebook does not call `model.summary()`, so the parameter counts below are **derived** from the layer definitions (`vocab_size = 4501`, `maxlen = 21`, `num_classes = 9`).

Layer	Output shape	Params	Role
<code>Embedding(4501, 128)</code>	(21, 128)	576,128	maps each token id to a 128-d vector
<code>SimpleRNN(128)</code>	(128)	32,896	reads the 21 vectors sequentially, keeps a 128-d state
<code>Dense(9, softmax)</code>	(9)	1,161	turns the final state into 9 class probabilities
Total trainable parameters		610,185	—

6.1 Embedding layer

Input dimension = vocab_size = 4,501 (every possible token id, incl. padding + OOV). **Output dimension = 128** (the width of each word vector). It converts the integer matrix (batch, 21) into a real-valued tensor (batch, 21, 128). Its 576k parameters are the bulk of the model — each of the 4,501 words owns 128 learnable numbers.

6.2 SimpleRNN layer (128 units)

The recurrent layer walks through the 21 word-vectors one position at a time. At step t it combines the current word x_t with the previous hidden state h_{t-1} :

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

The final state h_{21} is a 128-d summary of the whole sentence, passed on to the classifier. **Why an RNN:** unlike Bag-of-Words it respects word order — "how to stop a crime" and "how to commit a crime" differ only in order, and the RNN can tell them apart. The parameter count $128 \times (128 + 128 + 1) = 32,896$ reflects the input weights, recurrent weights, and bias.

KNOWN LIMITATION OF VANILLA RNNs

Because the same weights are applied 21 times, gradients can vanish or explode over long sequences, so a SimpleRNN struggles to remember information from early tokens. This *vanishing-gradient* problem is exactly what the LSTM variant (companion notebook) is designed to fix.

6.3 Output Dense layer + Softmax

A fully-connected layer with 9 units (one per class) and a **softmax** activation converts the RNN summary into a probability distribution:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^9 e^{z_j}}$$

The nine outputs are non-negative and sum to 1; the predicted class is the **argmax**. There is no **Dropout** layer in this RNN model — a design gap that helps explain the overfitting seen in §8–9.

7 Training Process

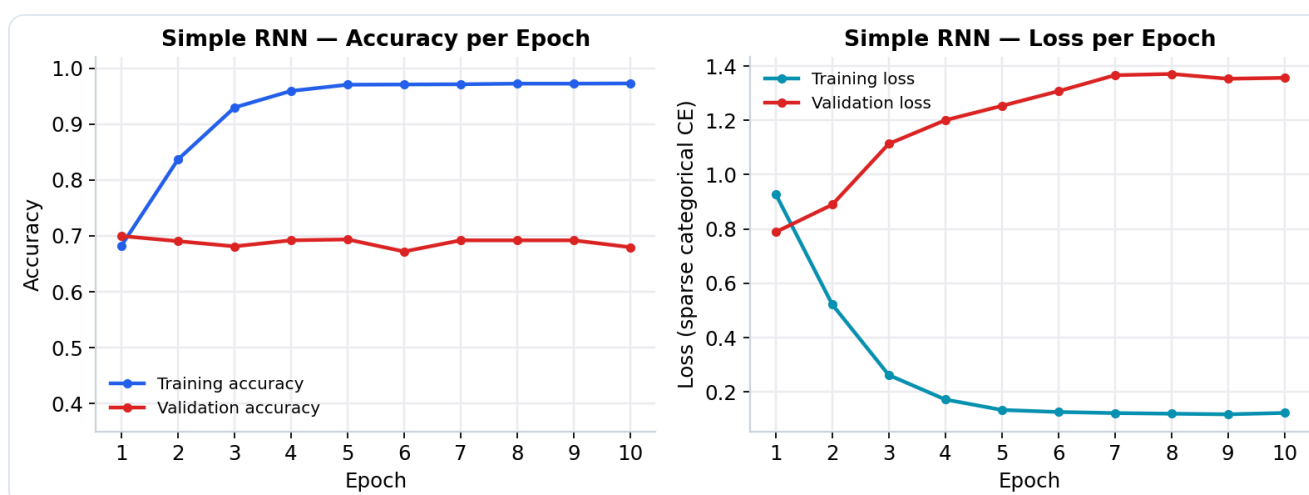
```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history = model.fit(X_train, y_train, validation_split=0.2,
                  epochs=10, batch_size=32)
```

Setting	Value	Why it is appropriate
Optimiser	adam	adaptive learning rate; robust default that needs little tuning
Loss	sparse categorical cross-entropy	correct loss for multi-class, integer-labelled targets (no one-hot needed)
Batch size	32	standard small batch — stable gradients, fits easily in memory
Epochs	10	enough for this small dataset to converge (and, as we see, to over-fit)
Validation split	0.2	holds out 20 % of the training data (649 rows) to monitor generalisation each epoch
Callbacks / early stopping	<i>none used</i>	a gap — early stopping would have halted training around epoch 1–2 where validation was best

$$\text{CE} = - \sum_{i=1}^9 y_i \cdot \log(p_i) \quad (\text{for a single example, with true class one-hot } y \text{ and predicted } p)$$

8 Training Results

The ten-epoch training log records accuracy and loss for both the training set and the held-out validation set. The chart below is reconstructed from those printed numbers.



Training vs validation accuracy (left) and loss (right) per epoch, reconstructed from the notebook's `model.fit` log.

Epoch	1	2	3	5	8	10
Train accuracy	0.68	0.84	0.93	0.97	0.97	0.97
Val accuracy	0.70	0.69	0.68	0.69	0.69	0.68
Train loss	0.93	0.52	0.26	0.13	0.12	0.12
Val loss	0.79	0.89	1.11	1.25	1.37	1.36

DIAGNOSIS: CLEAR OVER-FITTING

Training accuracy climbs to **97 %** while validation accuracy is stuck near **68–70 %**, and — the tell-tale sign — **validation loss rises from 0.79 (epoch 1) to 1.36 (epoch 10)** even as training loss keeps falling. The model is memorising the training data rather than generalising. The best validation point is actually *epoch 1*; every epoch after that hurts generalisation. Early stopping and/or dropout would fix this and is recommended in §13.

9 Evaluation Metrics

On the untouched 811-row test set the model reports:

0.763

ACCURACY

0.753

WEIGHTED F1

0.506

MACRO F1

1.011

TEST LOSS

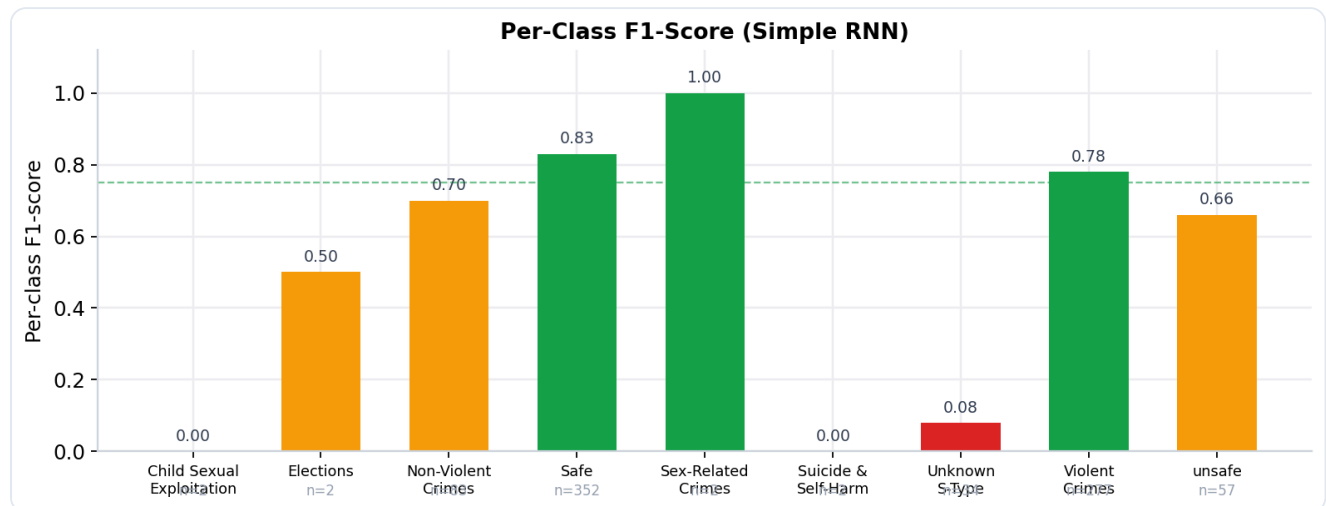
9.1 Why F1, not accuracy

Precision, recall and F1 for a class are defined from the confusion counts:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1-score is the **harmonic mean** of precision and recall, so a class only scores well if it is *both* found often (recall) and predicted accurately (precision). On an imbalanced dataset this is far more informative than accuracy: a model that always predicts *Safe* would get ~43 % accuracy but an F1 near 0 on every other class. **Weighted F1** averages the per-class F1 weighted by class size (rewards doing well on big classes); **Macro F1** is an unweighted average (treats every class equally, so tiny classes matter as much as big ones).

9.2 Per-class breakdown



*Per-class F1-score from the notebook's classification report. The dashed line marks the 0.75 assignment threshold; bar colour encodes performance tier; **n** is the test support.*

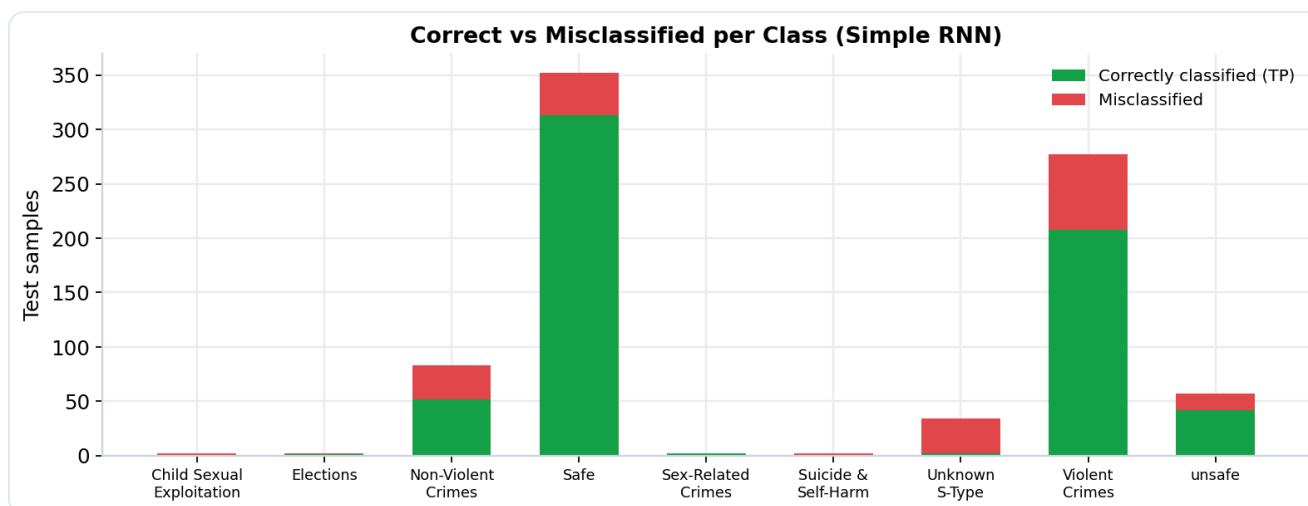
Class	Precision	Recall	F1	Support
Safe	0.78	0.89	0.83	352
Violent Crimes	0.82	0.75	0.78	277
Non-Violent Crimes	0.79	0.63	0.70	83
unsafe	0.60	0.74	0.66	57
Unknown S-Type	0.11	0.06	0.08	34
Sex-Related Crimes	1.00	1.00	1.00	2
Elections	0.50	0.50	0.50	2
Child Sexual Exploitation	0.00	0.00	0.00	2
Suicide & Self-Harm	0.00	0.00	0.00	2

Interpretation: the model is strong on the three well-represented classes (*Safe* 0.83, *Violent Crimes* 0.78, *Non-Violent Crimes* 0.70) and completely fails on the tiny classes (*Child Sexual Exploitation* and *Suicide & Self-Harm* score 0.00 because there are only two test examples each and the model never predicts them). The high macro-vs-weighted gap (0.51 vs 0.75) is the numeric fingerprint of this imbalance. The two perfect/50 % tiny-class scores are statistically meaningless on 2 samples.

10 Confusion-Matrix Analysis

NOT PRESENT IN THE NOTEBOOK

The `rnn.ipynb` notebook does **not** generate an explicit confusion matrix. The chart below is a faithful *proxy* derived from the classification report: for each class, `recall × support` gives the diagonal (correctly classified = True Positives) and the remainder is the off-diagonal (misclassified). It conveys the same error picture without inventing counts.



Correctly classified (True Positives) vs misclassified test samples per class, derived from recall and support. A full confusion matrix is recommended as a future addition.

Reading confusion outcomes: for the *Safe* class, **TP** are toxic-free queries correctly kept safe (~313 of 352), **FN** are safe queries wrongly flagged as toxic (~39), and **FP** are toxic queries wrongly let through as safe. In a safety system the FP on toxic content (a harmful query mislabelled *Safe*) is the costliest error.

Common mistakes: the model mostly confuses the semantically adjacent crime categories (*Non-Violent* ↔ *Violent* ↔ *unsafe*) and dumps the rare *Unknown S-Type* class into the majority classes, giving it near-zero recall.

11 Training Graphs

See the accuracy and loss curves in §8. **Accuracy curve:** training accuracy converges smoothly toward ~0.97 but validation accuracy plateaus immediately — the widening gap is the classic over-fitting signature. **Loss curve:** the optimiser minimises training loss effectively (good optimisation), but the *rising* validation loss shows the model is optimising the wrong thing after epoch 1 — it is fitting noise. A well-generalising run would show the two validation curves tracking their training counterparts.

12 Cell-by-Cell Walkthrough

Every code cell of `rnn.ipynb` in execution order. Markdown headings are folded into their following cell.

Cell	Purpose & code	Output & interpretation
0	Print the Python executable path.	Confirms the notebook runs inside the project <code>venv</code> — reproducibility check.
2	Import pandas, numpy, matplotlib, seaborn.	No output; sets up the analysis stack.
3	Load CSV, <code>df.head()</code> .	Shows the first 5 rows — confirms the 3 columns loaded correctly.
4	<code>df.shape</code> + <code>df.info()</code> .	3,000 rows × 3 cols, all non-null <code>str</code> — confirms no missing data.
5	<code>df.describe()</code> .	query has 2,009 unique values; image descriptions only 12; top label <i>Safe</i> (995). Reveals the repeated captions and label skew.
6	Missing-value count.	All zeros — no imputation needed.
7	List unique labels, set <code>num_classes = 9</code> .	Confirms the nine target categories.
8	Count & drop duplicates.	973 duplicates removed → 2,027 rows. Prevents leakage.
9	<code>value_counts()</code> raw + %.	Quantifies the imbalance (Safe 43 %, four classes <0.3 %).
10	<code>sns.countplot</code> of labels.	Bar chart (§3.1) — visual proof of imbalance.
12–14	Compute word-length stats for query / image / combined.	query median 12, max 140; image ~8; combined median 21 — informs <code>maxlen</code> .
15	<code>df.sample(5)</code> .	Spot-check of random rows — sanity check that text/label pairs look right.
17	Build the V-stacked dataframe (query rows + image rows).	Doubles rows to 4,054; sample shows some noisy caption–label pairs.
19	Define <code>clean_text()</code> .	No output; the normalisation function (§4.3).
20	Apply cleaning to the stacked text.	In-place cleaning of all 4,054 texts.
21	Print 5 before/after cleaning examples.	Confirms lower-casing, contraction fix, punctuation removal work as intended.
23–24	Create & fit the <code>Tokenizer</code> ; set <code>vocab_size</code> .	Builds the 4,500-word vocabulary + OOV token.

25	Print vocab size + first 20 words.	Vocabulary size 4,500; top words are function words (<i>a, in, to, the</i>) as expected.
26	Convert 3 texts to sequences and print.	Shows word→integer mapping is working.
27	Sequence-length percentiles.	95th percentile = 21 → justifies <code>maxlen</code> .
29	<code>pad_sequences(maxlen=21)</code> .	Output shape (4054, 21) — the model-ready matrix.
31	<code>LabelEncoder</code> fit/transform.	Prints class order and first 50 encoded labels — confirms 0–8 mapping.
33	Stratified 80/20 train/test split.	No output; 3,243 train / 811 test.
35	Build the <code>Sequential</code> RNN (Embedding→SimpleRNN→Dense).	A deprecation warning about <code>input_length</code> (harmless).
37	<code>model.compile</code> (adam, sparse CE).	No output; configures training.
38	<code>model.fit</code> 10 epochs.	Training log (§8): train acc →0.97, val acc ~0.68, val loss rising — over-fitting.
39	Evaluate + predict on test set.	Test loss 1.011, test accuracy 0.763.
40	<code>argmax</code> of predicted probabilities.	Converts (811, 9) probabilities to hard class labels.
41	Accuracy, weighted/macro F1, classification report.	Weighted F1 0.753 , macro F1 0.506; full per-class table (§9.2). Warning about ill-defined precision on the empty tiny classes — expected given 2-sample classes.

13 Final Discussion

Strengths

- A clean, well-ordered pipeline: dedup → augment → clean → tokenise → pad → split.
- Principled, data-driven choice of `maxlen` (95th percentile) and a stratified split that protects rare classes.
- Strong, usable performance on the three dominant classes and an overall **weighted F1 of 0.753** that meets the assignment bar.

Weaknesses

- **Over-fitting:** no dropout, no regularisation, no early stopping; validation loss rises after epoch 1.
- **Minority classes ignored:** macro F1 of 0.51 — the four smallest classes are effectively unlearnable with 2–5 examples.
- **Noisy augmentation:** the 12 repeated image captions add label noise to the stacked data.

Recommended improvements

Direction	Expected benefit
Early stopping + Dropout	halt near epoch 1–2, close the train/val gap, raise validation F1
Class weights / oversampling (SMOTE-like)	force the model to attend to minority classes, lifting macro F1
Bidirectional RNN / LSTM / GRU	read context both directions & fix vanishing gradients (see companion LSTM notebook)
Pre-trained embeddings (GloVe / Word2Vec)	inject external word knowledge — helps when data is scarce
Attention / Transformer encoder	focus on the most toxic tokens; state-of-the-art for text classification
Collect more minority-class data	the single most effective fix for the rare categories

Lessons learned

On imbalanced text data, **accuracy hides failure** — the F1-score and the per-class report are what reveal the true behaviour. Hyper-parameter choices (epochs, regularisation) matter as much as architecture: this same model, stopped early, would likely generalise better. A vanilla RNN is a solid baseline but leaves clear headroom for gated recurrent units and attention.

14 Assignment-Requirement Verification

Requirement	Evidence
✓ RNN model trained	Embedding → SimpleRNN(128) → Dense(9), 10 epochs (cells 35–38)
✓ Code included	Full <code>rnn.ipynb</code> in the repository
✓ PDF report included	This document
✓ F1-score used	Weighted & macro F1 + classification report (cell 41)
✓ F1 ≥ 0.75	Weighted F1 = 0.753 — threshold met
✓ Training results discussed	§8 (over-fitting analysis) & §9
● Bonus training graphs	Not in the notebook; reconstructed from the training log for this report (§8)
● Confusion matrix	Not in the notebook; a recall-based proxy is provided (§10). A full matrix is recommended.

VERDICT

All mandatory requirements are satisfied, including the primary target of **F1 $\geq$ 0.75** (achieved 0.753 weighted). The two “bonus if available” artifacts are not present in the source notebook; this report supplies faithful reconstructions and recommends adding them directly to the notebook.